

Operators in C

- **C Operators** are symbols that represent operations to be performed on one or more operands.
- C provides a wide range of operators, which can be classified into different categories based on their functionality.
- **Operators** are used for performing operations on variables and values.
- **Operators** can be defined as the symbols that help us to perform specific mathematical, relational, bitwise, conditional, or logical computations on operands.
- In other words, we can say that an operator operates the operands. For example, '+' is an operator used for addition, as shown below:

```
c = a + b;
```

Here, '+' is the operator known as the addition operator, and 'a' and 'b' are operands.

- The addition operator tells the compiler to add both operands 'a' and 'b'. The functionality of the C programming language is incomplete without the use of operators.

Types of Operators in C

C has many built-in operators and can be classified into 6 types:

- **Arithmetic Operators**
- **Relational Operators**
- **Logical Operators**
- **Bitwise Operators**
- **Assignment Operators**
- **Misc Operators**

➤ Arithmetic Operations in C

These operators are used to perform arithmetic/mathematical operations on operands. Examples: (+, -, *, /, %, ++, --). Arithmetic operators are of two types:

- **Binary Arithmetic Operators in C**

The C binary arithmetic operators operate or work on two operands. C provides **5** Binary Arithmetic Operators for performing arithmetic functions which are as follows:

Operator	Name of the Operator	Arithmetic Operation	Syntax
+	Addition	Add two operands.	$x + y$
-	Subtraction	Subtract the second operand from the first operand.	$x - y$
*	Multiplication	Multiply two operands.	$x * y$
/	Division	Divide the first operand by the second operand.	x / y
%	Modulus	Calculate the remainder when the first operand is divided by the second operand.	$x \% y$

- **Unary Arithmetic Operators in C**

The unary arithmetic operators operate or work with a single operand. In C, we have two unary arithmetic operators which are as follows:

Operator	Symbol	Operation	Implementation
Decrement Operator	—	Decreases the integer value of the variable by one.	–h or h–
Increment Operator	++	Increases the integer value of the variable by one.	++h or h++
Unary Plus Operator	+	Returns the value of its operand.	+h
Unary Minus Operator	–	Returns the negative of the value of its operand.	-h

Examples of C Arithmetic Operators

C Program to find the area and perimeter of a rectangle

```
#include <stdio.h>
int main()
{
// declaring dimensions of the rectangle
int length = 10;
int breadth = 5;
```

```
// declaring variables to store the results
int area, perimeter;

// calculating area
area = length * breadth;

// calculating perimeter
perimeter = 2 * (length + breadth);

// printing results
printf("Area = %d\nPerimeter = %d", area, perimeter);

return 0;
}
```

Output:

```
Area = 50
Perimeter = 30
```

➤ Relational Operators in C

These are used for the comparison of the values of two operands. For example, checking if one operand is equal to the other operand or not, whether an operand is greater than the other operand or not, etc. Some of the relational operators are (==, >= , <=)(See [this](#) article for more reference).

```
int a = 3;
int b = 5;
cout<<(a < b);
```

➤ Logical Operator in C

Logical Operators are used to combining two or more conditions/constraints or to complement the evaluation of the original condition in consideration. The

result of the operation of a logical operator is a Boolean value either **true** or **false**.

We have 3 logical operators in the C language:

- **Logical AND (&&)**
- **Logical OR (||)**
- **Logical NOT (!)**

- **Logical AND Operator (&&)**

If both operands are non zero then the condition becomes true. Otherwise, the result has a value of 0. The return type of the result is int. Below is the truth table for the logical AND operator.

X	Y	X && Y
1	1	1
1	0	0
0	1	0
0	0	0

- **Logical OR Operator (||)**

The condition becomes true if any one of them is non-zero. Otherwise, it returns false i.e., 0 as the value. Below is the truth table for the logical OR operator.

X	Y	X Y
1	1	1
1	0	1
0	1	1
0	0	0

- **Logical NOT Operator (!)**

If the condition is true then the logical NOT operator will make it false and vice-versa. Below is the truth table for the logical NOT operator.

X	!X
0	1
1	0

➤ Bitwise Operators in C

- The Bitwise operators are used to perform bit-level operations on the operands.
- The operators are first converted to bit-level and then the calculation is performed on the operands.
- Mathematical operations such as addition, subtraction, multiplication, etc. can be performed at the bit level for faster processing.

- For example, the **bitwise AND** operator represented as '**&**' in **C** takes two numbers as operands and does AND on every bit of two numbers.
- The result of AND is 1 only if both bits are 1(True).

```
int a = 5, b = 9; // a = 5(00000101), b = 9(00001001)
cout << (a ^ b); // 00001100
cout << (~a);    // 11111010
```

➤ Assignment Operators in C

- Assignment operators are used to assign value to a variable. The left side operand of the assignment operator is a variable and the right side operand of the assignment operator is a value.
- The value on the right side must be of the same data type as the variable on the left side otherwise the compiler will raise an error.

Different types of assignment operators are shown below:

Assignment Operators		Suppose X = 25, Y = 14
Operator	Expression	Description
=	X = Y	Assigns Y value to X
+=	X += Y	X = X + Y
-=	X -= Y	X = X - Y
*=	X *= Y	X = X * Y
/=	X /= Y	X = X / Y
%=	X %= Y	X = X % Y

C program to demonstrate working of Assignment operators:

```
#include <stdio.h>
int main()
{
    // Assigning value 10 to a
```

```
// using "=" operator
int a = 10;
printf("Value of a is %d\n", a);
// Assigning value by adding 10 to a
// using "+=" operator
a += 10;
printf("Value of a is %d\n", a);
// Assigning value by subtracting 10 from a
// using "-=" operator
a -= 10;
printf("Value of a is %d\n", a);
// Assigning value by multiplying 10 to a
// using "*=" operator
a *= 10;
printf("Value of a is %d\n", a);
// Assigning value by dividing 10 from a
// using "/=" operator
a /= 10;
printf("Value of a is %d\n", a);
return 0;
}
```

Output:

```
Value of a is 10
Value of a is 20
Value of a is 10
Value of a is 100
Value of a is 10
```

➤ Misc Operators in C ↪ sizeof & Ternary

Besides the operators discussed above, there are a few other important operators including **sizeof** and **? :** supported by the C Language.

Operator	Description	Example
sizeof()	Returns the size of a variable.	sizeof(a), where a is integer, will return 4.
&	Returns the address of a variable.	&a; returns the actual address of the variable.
*	Pointer to a variable.	*a;
? :	Conditional Expression.	If Condition is true ? then value X : otherwise value Y